

MACHINE LEARNING FOR SIGNAL PROCESSING

5-4-2025

Sriram Ganapathy

LEAP lab, Electrical Engineering, Indian Institute of Science,
sriramg@iisc.ac.in

.....
Viveka Salinamakki, Varada R.

LEAP lab, Electrical Engineering, Indian Institute of Science
.....

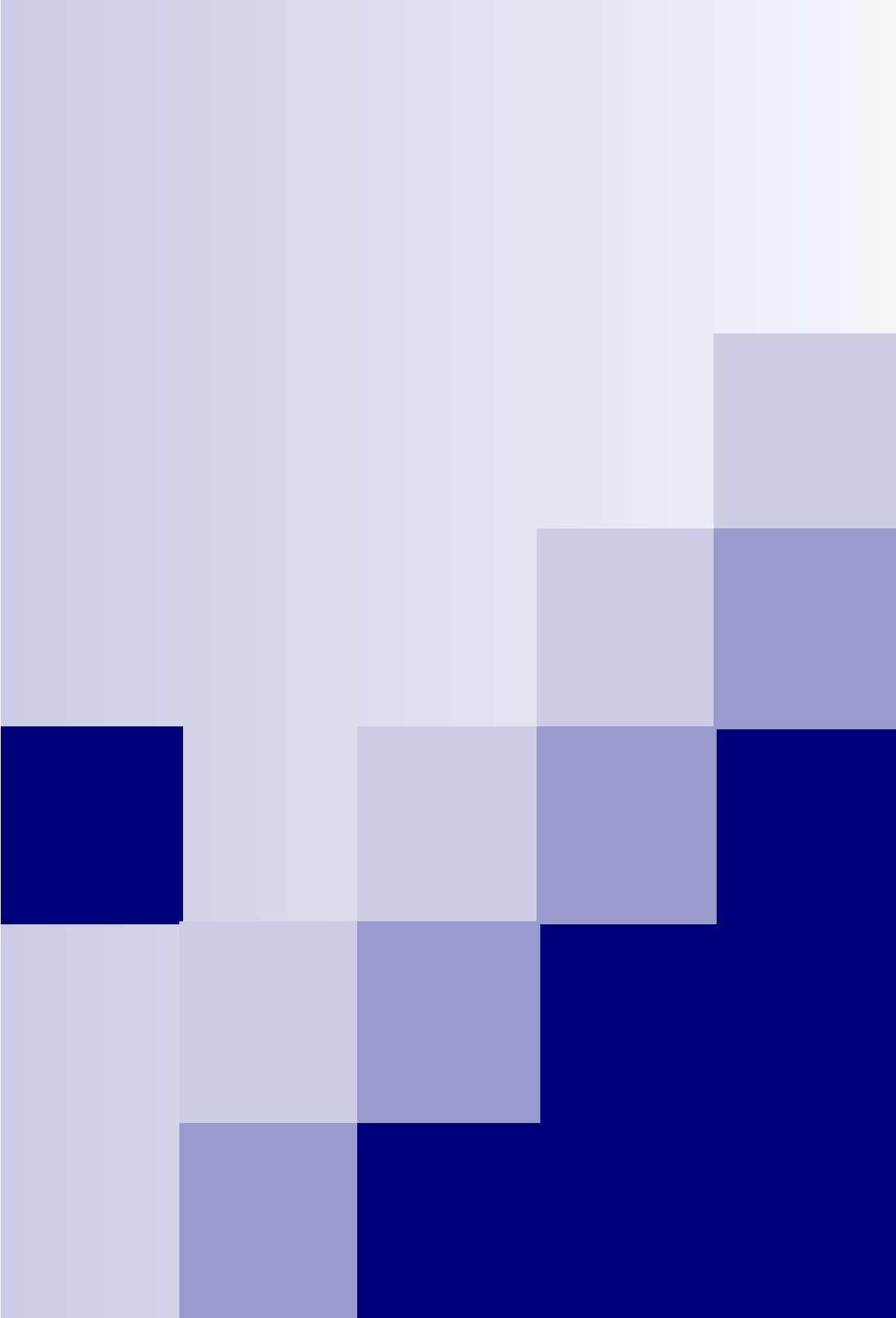
<http://leap.ee.iisc.ac.in/sriram/teaching/MLSP25/>



Graphical Models

Graphical Models

- What are graphs
 - * What are graphical models
- Directed and Undirected graphs
- Conditional independence



Hidden Markov Models

Adapted from
Dr Catherine Sweeney-Reed's
slides



Summary

- Introduction
- Description
- Central problems in HMM modelling
- Extensions
- Demonstration

Specification of an HMM

- *N* - number of states
 - $Q = \{q_1; q_2; \dots; q_T\}$ - set of states
- *M* - the number of symbols (observables)
 - $O = \{o_1; o_2; \dots; o_T\}$ - set of symbols

Specification of an HMM

- *A - the state transition probability matrix*
 - $a_{ij} = P(q_{t+1} = j | q_t = i)$
- *B- observation probability distribution*
 - $b_j(k) = P(o_t = k | q_t = j) \quad i \leq k \leq M$
- *π - the initial state distribution*



Specification of an HMM

Description

- Full HMM is thus specified as a triplet:
 - $\lambda = (A, B, \pi)$



Central problems in HMM modelling

■ Problem 1

Evaluation:

- Probability of occurrence of a particular observation sequence, $O = \{o_1, \dots, o_k\}$, given the model
- $P(O|\lambda)$
- Complicated – hidden states
- Useful in sequence classification



Central problems in HMM modelling

- Problem 2

Decoding:

- Optimal state sequence to produce given observations, $O = \{o_1, \dots, o_k\}$, given model
- Optimality criterion
- Useful in recognition problems



Central problems in HMM modelling

- Problem 3

Learning:

- Determine optimum model, given a training set of observations
- Find λ , such that $P(O|\lambda)$ is maximal

Problem 1: Naïve solution

Central
problems

- State sequence $Q = (q_1, \dots, q_T)$
- Assume independent observations:

$$P(O | q, \lambda) = \prod_{i=1}^T P(o_t | q_t, \lambda) = b_{q_1}(o_1) b_{q_2}(o_2) \dots b_{q_T}(o_T)$$

NB Observations are mutually independent, given the hidden states. (Joint distribution of independent variables factorises into marginal distributions of the independent variables.)

Problem 1: Naïve solution

- Observe that :

$$P(q \mid \lambda) = \pi_{q1} a_{q1q2} a_{q2q3} \cdots a_{qT-1qT}$$

- And that:

$$P(O \mid \lambda) = \sum_q P(O \mid q, \lambda) P(q \mid \lambda)$$

Problem 1: Naïve solution

- Finally get:

$$P(O | \lambda) = \sum_q P(O | q, \lambda) P(q | \lambda)$$

NB:

- The above sum is over all state paths
- There are N^T states paths, each 'costing' $O(T)$ calculations, leading to $O(TN^T)$ time complexity.

Problem 1: Efficient solution

Forward algorithm:

- Define auxiliary forward variable α :

$$\alpha_t(i) = P(o_1, \dots, o_t \mid q_t = i, \lambda)$$

$\alpha_t(i)$ is the probability of observing a partial sequence of observables $o_1, \dots, o_t$ such that at time t , state $q_t = i$

Problem 1: Efficient solution

- Recursive algorithm:

- Initialise:

- Calculate: $\alpha_1(i) = \pi_i b_i(o_1)$

(Partial obs seq to t AND state i at t)
x (transition to j at $t+1$) x (sensor)

- Obtain: $\alpha_{t+1}(j) = \left[\sum_{i=1}^N \alpha_t(i) a_{ij} \right] b_j(o_{t+1})$

Sum, as can reach j from
any preceding state

α incorporates partial obs seq to t

$$P(O | \lambda) = \sum_{i=1}^N \alpha_T(i)$$

Sum of different ways
of getting obs seq

Complexity is $O(N^2T)$

Problem 1: Alternative solution

Backward algorithm:

- Define auxiliary forward variable β :

$$\beta_t(i) = P(o_{t+1}, o_{t+2}, \dots, o_T \mid q_t = i, \lambda)$$

$\beta_t(i)$ – the probability of observing a sequence of observables $o_{t+1}, \dots, o_T$ given state $q_t = i$ at time t , and λ

Problem 1: Alternative solution

- Recursive algorithm:

- Initialise:

$$\beta_T(j) = 1$$

- Calculate:

- Terminate: $\beta_t(i) = \sum_{j=1}^N \beta_{t+1}(j) a_{ij} b_j(o_{t+1})$

$$p(O | \lambda) = \sum_{i=1}^N \beta_1(i) \quad t = T-1, \dots, 1$$

Complexity is $O(N^2T)$

Problem 2: Decoding

- Choose state sequence to maximise probability of observation sequence
- Viterbi algorithm - inductive algorithm that keeps the best state sequence at each instance

Problem 2: Decoding

Viterbi algorithm:

- State sequence to maximise $P(O, Q | \lambda)$:

$$P(q_1, q_2, \dots, q_T | O, \lambda)$$

- Define auxiliary variable δ :

$$\delta_t(i) = \max_q P(q_1, q_2, \dots, q_t = i, o_1, o_2, \dots, o_t | \lambda)$$

$\delta_t(i)$ – the probability of the most probable path ending in state $q_t = i$

Problem 2: Decoding

- Recurrent property:

To get state seq, need to keep track of argument to maximise this, for each t and j . Done via the array $\psi_t(j)$.

$$\delta_{t+1}(j) = \max_i (\delta_t(i) a_{ij}) b_j(o_{t+1})$$

- Algorithm:
 - 1. Initialise:

$$\delta_1(i) = \pi_i b_i(o_1) \quad 1 \leq i \leq N$$

$$\psi_1(i) = 0$$

Problem 2: Decoding

Central
problems

- 2. Recursion:

$$\delta_t(j) = \max_{1 \leq i \leq N} (\delta_{t-1}(i) a_{ij}) b_j(o_t)$$

- 3. Terminate: $\psi_t(j) = \arg \max_{1 \leq i \leq N} (\delta_{t-1}(i) a_{ij}) \quad 2 \leq t \leq T, 1 \leq j \leq N$

$$P^* = \max_{1 \leq i \leq N} \delta_T(i)$$

P^* gives the state-optimised probability

$$q_T^* = \arg \max_{1 \leq i \leq N} \delta_T(i)$$

Q^* is the optimal state sequence
($Q^* = \{q_1^*, q_2^*, \dots, q_T^*\}$)

Problem 2: Decoding

Central
problems

- 4. Backtrack state sequence:

$$q_t^* = \psi_{t+1}(q_{t+1}^*) \quad t + T - 1, T - 2, \dots, 1$$

$O(N^2T)$ time complexity

Problem 3: Learning

- Training HMM to encode obs seq such that HMM should identify a similar obs seq in future
- Find $\lambda=(A,B,\pi)$, maximising $P(O|\lambda)$
- General algorithm:
 - Initialise: λ_0
 - Compute new model λ , using λ_0 and observed sequence O
 - Then $\lambda \leftarrow \lambda$
 - Repeat steps 2 and 3 until:

$$\log P(O | \lambda) - \log P(O | \lambda_0) < d$$

Problem 3: Learning

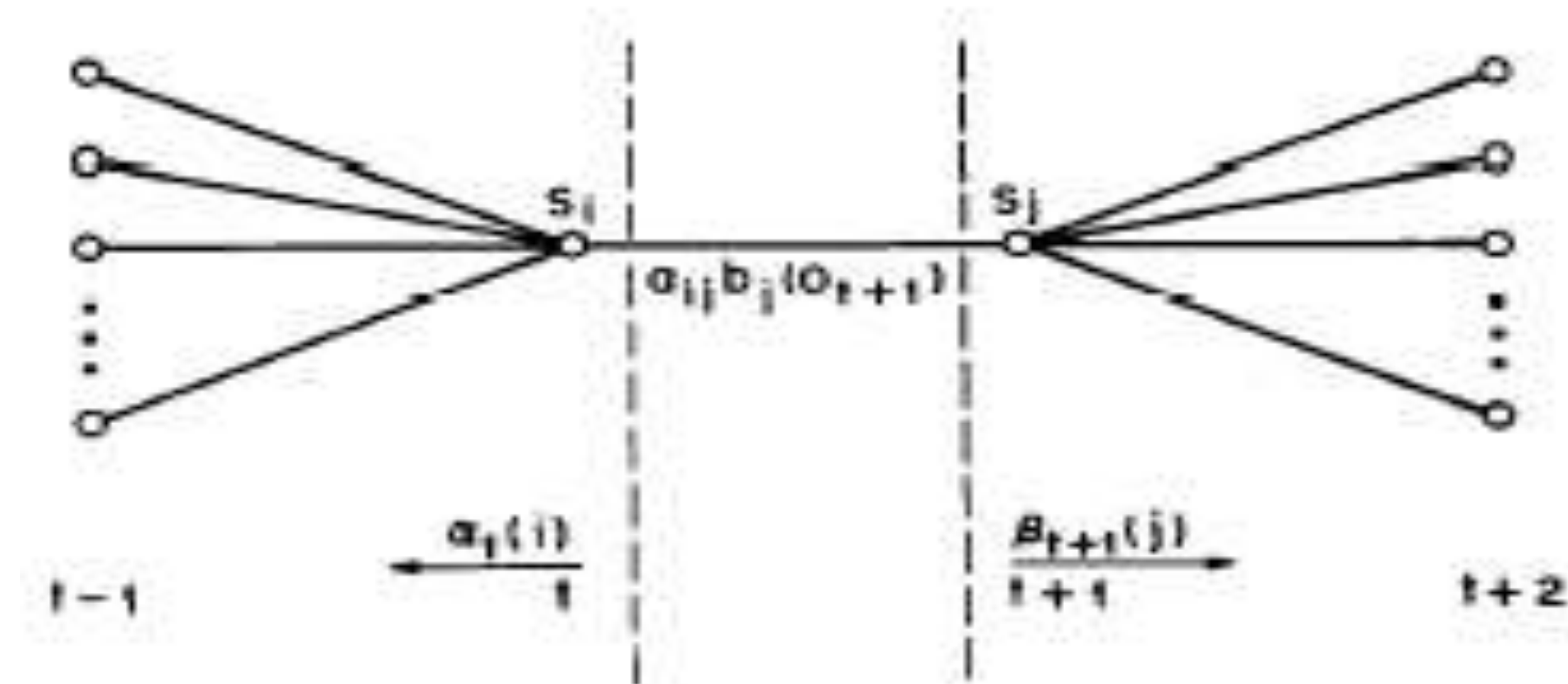
Step 1 of Baum-Welch algorithm:

- Let $\xi(i,j)$ be a probability of being in state i at time t and at state j at time $t+1$, given λ and O seq

$$\xi(i, j) = \frac{\alpha_t(i) a_{ij} b_j(o_{t+1}) \beta_{t+1}(j)}{P(O | \lambda)}$$
$$= \frac{\alpha_t(i) a_{ij} b_j(o_{t+1}) \beta_{t+1}(j)}{\sum_{i=1}^N \sum_{j=1}^N \alpha_t(i) a_{ij} b_j(o_{t+1}) \beta_{t+1}(j)}$$

Problem 3: Learning

Central
problems



Operations required for the computation
of the joint event that the system is in state
 S_i and time t and State S_j at time $t+1$

Problem 3: Learning

- Let $\gamma_t(i)$ be a probability of being in state i at time t , given O

$$\gamma_t(i) = \sum_{j=1}^N \xi_t(i, j)$$

- $\sum_{t=1}^{T-1} \gamma_t(i)$ - expected no. of transitions from state i
- $\sum_{t=1}^{T-1} \xi_t(i, j)$ - expected no. of transitions $i \rightarrow j$

Problem 3: Learning

Central
problems

Step 2 of Baum-Welch algorithm:

- $\hat{\pi} = \gamma_1(i)$ the expected frequency of state i at time $t=1$

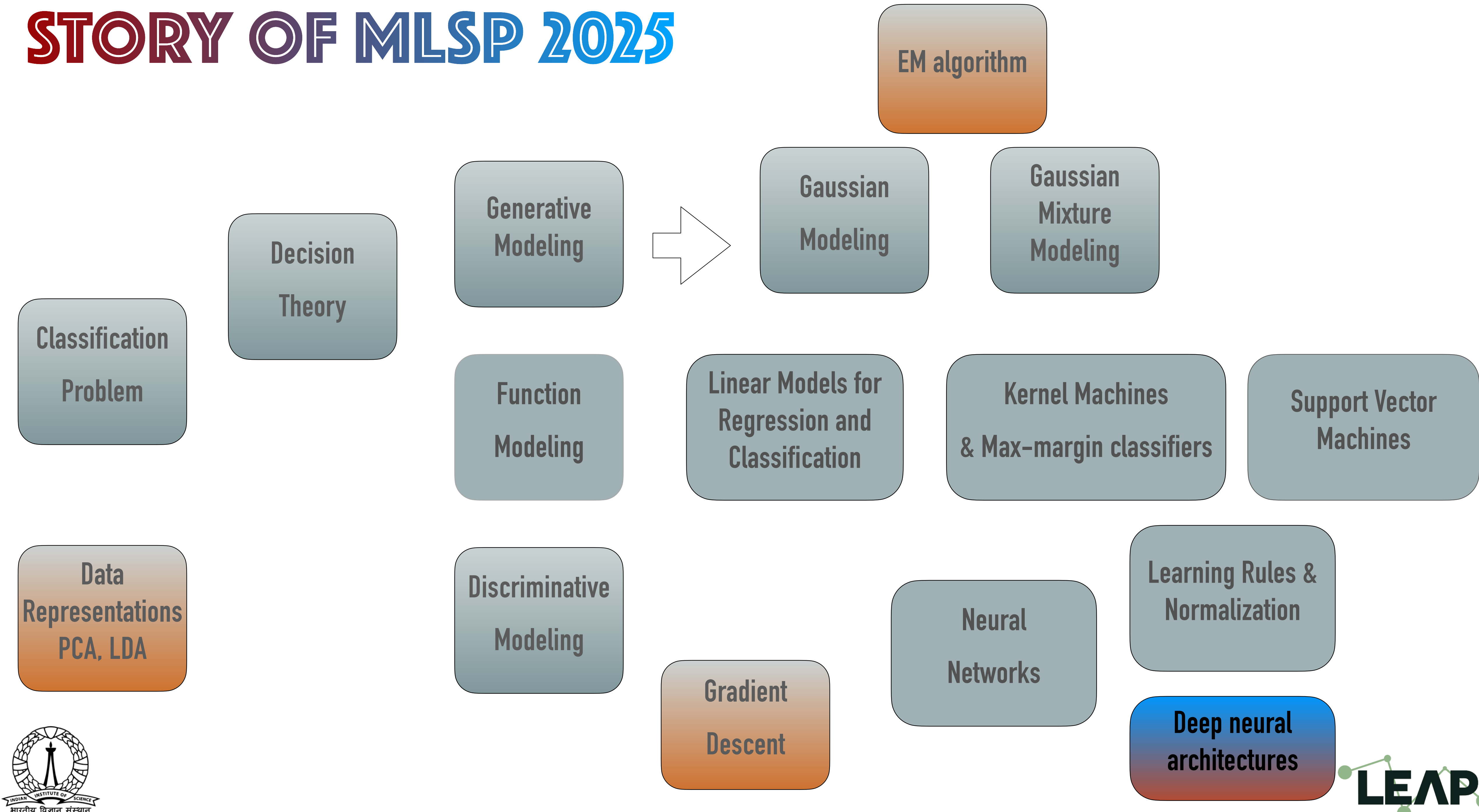
- $\hat{a}_{ij} = \frac{\sum \xi_t(i, j)}{\sum \gamma_t(i)}$ ratio of expected no. of transitions from state i to j over expected no. of transitions from state i

- $\hat{b}_j(k) = \frac{\sum_{t, o_t=k} \gamma_t(j)}{\sum \gamma_t(j)}$ ratio of expected no. of times in state j observing symbol k over expected no. of times in state j

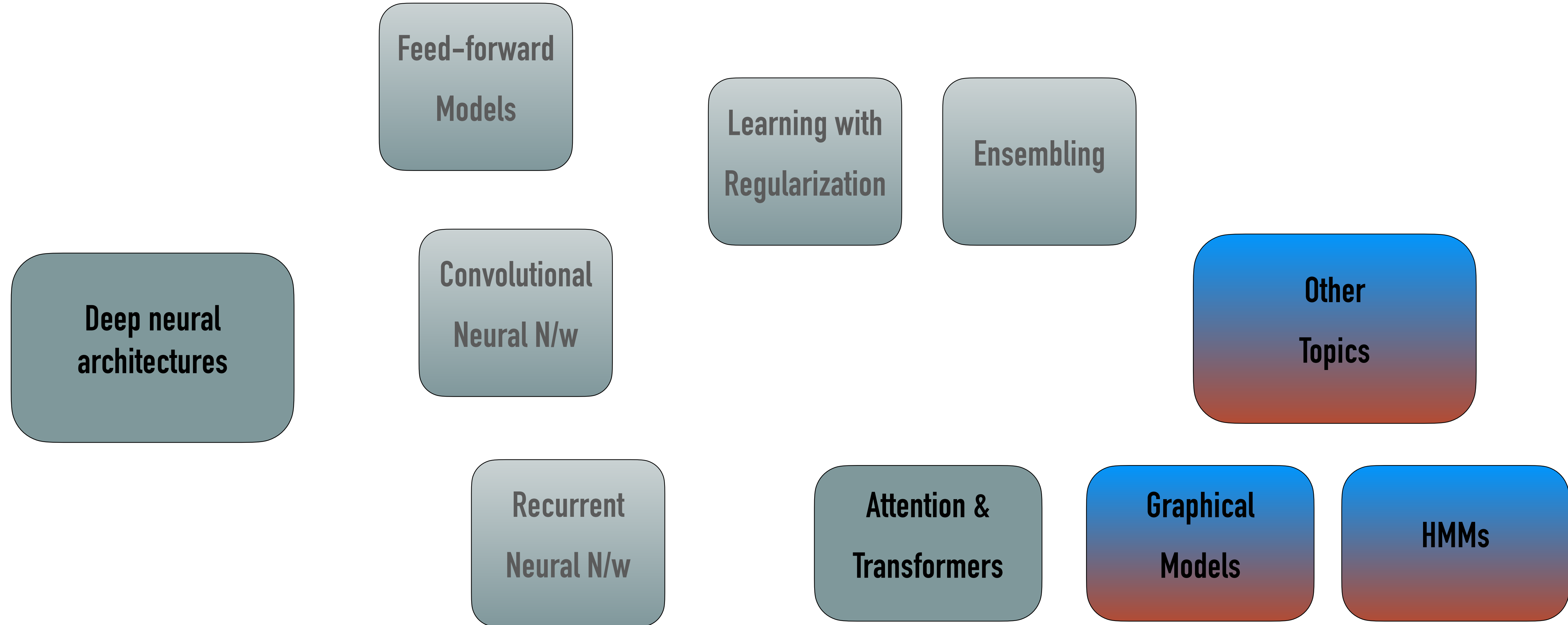
Problem 3: Learning

- Baum-Welch algorithm uses the forward and backward algorithms to calculate the auxiliary variables α, β
- B-W algorithm is a special case of the EM algorithm:
 - E-step: calculation of ξ and γ
 - M-step: iterative calculation of $\hat{\pi} \quad \hat{a}_{ij} \quad \hat{b}_j(k)$
- Practical issues:
 - Can get stuck in local maxima
 - Numerical problems – log and scaling

STORY OF MLSP 2025



STORY OF MLSP 2025



CONTENT DELIVERY

Theory
and Mathematical
Foundation

Implementation
and Understanding

Intuition and
Analysis

COURSE STRUCTURE (ROUGH SCHEDULE)

- ✓ ❖ Introduction to real world data and signals - text, speech, image, video.
- ✓ ❖ Dimensionality reduction - principal components, linear discriminants.
- ✓ ❖ Decision theory for pattern recognition, ML and MAP methods, Bias-variance trade-off, model assessment, cross-validation, estimating generalization error.
- ✓ ❖ Generative modeling and density estimation - Gaussian and mixture Gaussian models, kernel density estimators, hidden Markov models. Expectation Maximization.
- ✓ ❖ Linear regression and kernel methods. Regularization methods.
 - ❖ Discriminative modeling - support vector machines, **decision trees and random forest classifiers**, bagging and boosting.
- ✓ ❖ Neural networks: gradient descent optimization and back propagation, regularization in neural networks, dropout. normalization methods.
- ✓ ❖ Introduction to deep learning - feedforward, convolutional and recurrent networks, practical considerations in deep learning.
- ✓ ❖ Introduction to transformer models - self and cross attention, encoder and decoder architectures, autoregressive decoding.

DATES OF VARIOUS RITUALS

- ❖ 3 Assignments spread over 3 months (roughly one assignment every 3 weeks). ✓
- ❖ February second half - Midterm ✓
- ❖ February 4th week - project topic and team finalization and proposal submission. [1 and 2 person teams]. ✓
- ❖ March 3rd week - Project MidTerm Presentations. ✓
- ❖ April 3rd week - Final Exam
- ❖ April last week - Project Final Presentations

THANK YOU

Sriram Ganapathy and TA team
LEAP lab, C328, EE, IISc
sriramg@iisc.ac.in

